

Game Server Development

Installation

```
npm i -g typescript ts-node @types/node
npm i @slotify/gdk
```

To install private `@slotify/slotify` package you need to request private key. One of the ways to configure it is to create `.npmrc` file with the following content where `XXX` should be replaced with private key.

```
//npm.pkg.github.com/:_authToken=XXX
@slotify:registry=https://npm.pkg.github.com/
```

After configuring token you can install and start the service.

Running

```
GAMES_PATH=games/*/index.ts ts-node node_modules/@slotify/gdk/lib/index.js
```

Docker

Sample `Dockerfile` (remember to replace `PROVIDER_NAME` with name of your provider):

```
FROM node:24-slim

RUN mkdir -p /usr/src/tequity/games

WORKDIR /usr/src/tequity/games

COPY . .

RUN npm install --production
RUN npm run build

EXPOSE 8080

ENV PROVIDER=PROVIDER_NAME
ENV GAMES_PATH=lib/games/*/index.js
CMD ["node", "node_modules/@slotify/gdk/lib/index.js"]
```

Folder structure

Games should be possible to discover under path specified under `GAMES_PATH` (this path points to path after TypeScript->JavaScript compilation into `lib` directory). Default structure for `GAMES_PATH=lib/games/*/index.js` would be:

```
games/game1/index.ts
games/game2/index.ts
games/gameX/index.ts
```

Game Server

To set up a game you need to create file `index.ts` in the directory or subdirectory of path specified under `GAMES_PATH`. That file should expose object implementing `IGame` interface. Game needs to have its unique `name` assigned (optionally name can be an array of names - in specific cases when provider wants to run multiple identical game instances under different names).

Below example presents very simple "double or nothing" game - for any given bet you have 50% to double it and 50% of chance to lose your bet.

```
export const server: IGame = {
  name: "double-or-nothing",
  bets: {
    "main": {available: [1, 2, 5], default: 2, maxWin: 2, coin: 1},
  },
  play: ({bet}, random) => {
    const win = random(2) === 0 ? 0 : bet * 2;
    return {win};
  },
};

export default server;
```

Play

Play function is triggered for each request to the server. It is provided with object with the following params:

- `bet` - initial bet
- `sideBet` - (optional) side bet. For consecutive actions with bet included. Must pass custom validation
- `action` - action provided by a player. Initial action is `main`
- `params` - (optional) additional parameters coming from game client
- `state` - persistent data between rounds
- `coin` - coin defined in bet section
- `config` - config for given variant
- `variant` - (optional) game variant

Second argument is the random numbers generator:

- `random: (limit?: number) => number` - see [Random Numbers Generator](#).

Additionally there is third parameter that for advanced action validation:

- `betLimits: IBetLimits` - see [Bet Limits](#).

It is expected to return the following object:

- `win` - player's money won
- `data` - any gameplay data
- `state` - any data to be sent to the next play (Fields starting with underscore (`_`) are only for internal use in the server and will not be sent to game client)
- `next` - array with next actions
- `feed` - object to store as a game feed (e.g. hot numbers)

Bets

Bets object contains bets definition for each available *initial* commands.

Base bet action `SHOULD` be called `main`

- `available` : available bets in base currency (either array of specified items or object with `min`, `max` and `step`)
- `default` : default bet in base currency
- `maxWin` : theoretical maximum win multiplier used to calculate max exposure and max bet per operator
- `coin` : coin bet. It is only used to calculate win multiplier of base bet (`main` action). So important to keep proportion i.e. coin of `main` is 1 and coin of `bonus` 100 (if `bonus` is x100 base bet).
- `validate` : if `true` it will call validate method even on initial bet

```
export const server: IGame = {
  bets: {
    "main": {available: [0.1, 1, 2, 5], default: 2, maxWin: 200, coin: 10},
    "bonus": {available: {min: 100, max: 200, step: 25}, default: 100, maxWin: 5, coin: 1000},
  },
};
```

Config

Returns config for specified variant. If game supports only one variant then `variant` param can be omitted. Fields starting with underscore (`_`) are only for internal use in the server and will not be sent to game client.

```
export const server: IGame = {
  config: (variant) => ({
    symbols: ["high1", "high2", "high3"],
    mathValue: variant === "a" ? 1 : 2,
    _privateValue: "abc",
  }),
};
```

Validate

Validate method is custom validation for side bets and optionally also for initial bets (if `validate` is set to true in bets definitions).

It **MUST NOT** return `true` by default as that can lead to unwanted side bets.

`request` object includes:

- `bet` - initial bet
- `sideBet` - (optional) side bet
- `action` - action provided by a player. Initial action is `main`
- `params` - (optional) additional parameters coming from game client
- `state` - persistent data between rounds
- `coin` - coin defined in bet section
- `variant` - (optional) game variant

Second argument is:

- `betLimits: IBetLimits` - see [Bet Limits](#).

```
export const server: IGame = {
  validate: (request, betLimits) => {
    if (request.bet < betLimits.minBet) return false;
    if (request.bet > betLimits.maxBet) return false;
    return true;
  }
};
```

```
    },
  };

```

Cheats

Cheats help you jump to desired outcome. The RGS runs the rounds until the given cheat function is satisfied. To avoid infinite loops there cheat is rejected if it's not satisfied after certain number of simulations (exact number depends on RGS configuration).

```
export const server: IGame = {
  cheats: {
    "main": {
      "win": wager => wager.win > 0,
    },
    "bonus": {
      "big win": wager => wager.win / wager.bet > 20,
    },
  },
};

```

Action

For multi steps games it is required that RGS is able to complete the game in case player leaves the open round and doesn't make `play` requests. To get `action` and `params` (RGS placing `sideBet` on player's account is not allowed) to continue the game, RGS will call game's `action` method and pass latest `wager` as argument. `random` is also passed as argument in case action is non-deterministic.

```
export const server: IGame = {
  action({next, config}, random) {
    if (next.includes("takeWin")) {
      return {action: "takeWin"};
    }

    return {
      action: "continue",
      params: {
        position: random(config.numberOfTiles),
      },
    };
  },
};

```

Simulate

For multi steps games the Simulation Engine needs to be able to emulate player's decisions in order to play through the rounds. Player's decisions can be based `strategy`, `wagers` for played for a given round so far, `state` and `random`. Method can return any of the player input values: `action`, `params`, `bet`, `sideBet` - these will be used to compose next player `play` request.

```
export const server: IGame = {
  simulate({strategy, wagers}, random) {
    const lastWager = wagers[wagers.length - 1];
    if (lastWager && lastWager.next.includes("double") && strategy === "alwaysDouble") {
      return {
        action: "double",
        params: {call: "heads"},
        sideBet: sumOfBets(wagers),
      };
    }
  },
};

```

```

    }
    if (lastWager) {
        return {action: "takeWin"};
    }
},
};

```

Evaluate

Optional method is used to pass any data to promo module i.e. evaluating achievement or checking if jackpot was won.

```

export const server: IGame = {
    evaluate(type: string, wagers: IWager[], data: any) {
        switch (type) {
            case "jackpot":
                return {jackpotWon: wagers[0].data.jackpotWon};
            case "achievement":
                return {jackpotWon: wagers[0].win > 0};
        }
    },
};

```

Regulatory - Portugal

A specific case for using `evaluate` is to provide regulatory information required in certain jurisdictions.

To ensure compliance with regulations for game launches in Portugal, it is mandatory to include `sm_result` and `descr_ap` strings that describe each game round (other "AJOG file" fields are optional). The platform can transmit these values to Portuguese operators, provided the `regulatory-pt` evaluation is implemented and the returned data adheres to the format `{ "sm_result": string, "descr_ap": string }`:

```

export const server: IGame = {
    evaluate(type: string, wagers: IWager[]) {
        switch (type) {
            case "regulatory-pt":
                return {
                    "sm_result": createSmResult(wagers), // game's custom implementation
                    "descr_ap": createDescrAp(wagers), // game's custom implementation
                };
            // other evaluate cases
        }
    },
};

```

Defining stats

To define what stats you want to extract you need to create `stats` section in your game server definition.

```

export const server: IGame = {
    stats: {
        "RTP": new RTP(),
        "RTP (bonus)": new RTP<ISpin[]>().filter(wagers => wagers[0].bonus),
        "HF": new HitPercentage(wagers => sumOfWins(wagers) > 0),
    },
};

```

You have following built-in stats:

- `Average` - average value
- `AverageWin` - average win
- `Balance` - final balance
- `HitFrequency` - hit frequency 1 in ...
- `HitPercentage` - hit percentage X%
- `Iteration` - number of iterations
- `Max` - max value
- `MaxWin` - max win
- `Median` - median (warning: very slow on longer simulations)
- `MedianWin` - median win (warning: very slow on longer simulations)
- `RTP` - RTP (Return To Player)
- `TimesWin` - times win in last wager
- `Variance` - variance
- `Confidence Interval` - confidence interval for given confidence level

Each class has `filter` and `map` functions to help extracting the value you want.

Custom stats

If you need to define your custom stats you can do that by extending `Stats` class. Due to the fact simulation engine runs in multicore environment you need to implement simple map-reduce.

```
export default class CustomStats<TData = any, TState = any> extends Stats<TData, TState> {

  protected evenWins: number;

  constructor() {
    super();
  }

  value() { //returns pure value
    return this.evenWins;
  }

  message() { //returns formatted message
    return this.value()
      .toLocaleString(undefined, {maximumFractionDigits: 2})
      .replace(/,/g, " ");
  }

  protected processWagers(wagers) { //processes the wagers
    if (sumOfWins(wagers) % 2 === 0) {
      this.evenWins++;
    }
  }

  mapResults() { //send data to master core
    return {evenWins: this.evenWins};
  }

  reduceResults(result) { //received data from slave core
    this.evenWins += result.evenWins;
  }

  clearResults() { //clear
    this.evenWins = 0;
  }
}
```

Prove Fairness

This method is required for games intended to be played in **provably fair** mode. It uses cryptographic methods for random numbers generation.

This doesn't affect gameplay implementation itself, but this type of games requires proving that game outcomes deterministically depend on player's and casino's seeds.

GDK handles proving that random numbers are generated from provided seeds. But game still needs to specify a sequence of randomizations and provide mapping from random numbers to "game events" specific for the game (e.g. randomising a sequence of "left"/"right" directions of a Plinko ball dropping).

To achieve the above, `proveFairness` method receives randomization builder that registers what random number's `limit` a game is going to require and a `gameEventProducer` that will map the actual random number (passed by the GDK based on `limit` provided) into a game specific event: `export type IRandomizationBuilder = (limit: number, gameEventProducer: (randomNumber: number) => any) => void;`

Optionally a game can require client to send additional data (representing player choices or specific game configuration) that affects gameplay (e.g. number of rows player picked in Plinko).

```
proveFairness(addRandomization: IRandomizationBuilder, data: {plinkoRows: number}) {
  for (let i = 0; i < data.plinkoRows; i++) {
    addRandomisation(2, randomNumber => {
      return {
        direction: randomNumber ? "left" : "right",
      };
    });
  }
},
```

Random Numbers Generator

Function `random(limit?: number): number` is used as an argument in game methods and serves as a Random Number Generator that returns values from `32-bit` range.

When `limit` is provided, the function returns integer within the range `[0, limit)` with **uniform distribution**. `limit` must be a positive integer greater than zero and not exceeding `232 = 4294967296`. If no limit is specified, the function will use default maximal limit `232` thus returning a random number within the full 32-bit range `[0, 232)`. An error will be thrown if the limit is not a positive integer or is greater then `232`.

The underlying RNG algorithm can be selected via `--rngAlgorithm` flag of the simulation engine, or via `RNG_ALGORITHM` environment variable of the games service. There are currently two RNG algorithms available to choose from:

- `isaac` (default, certified)
- `mersenne-twister`

NOTE: When provable fairness is enabled for a game, numbers are generated via cryptographic methods using player's and casino's seeds. However, from game perspective this doesn't affect `random` function specification/usage.

Simulation Engine

Simulation engine makes it fast & easy to run simulations of your game server.

```
npx slotify-gdk stats -g games/my_game -i 5m
```

Options:

-g, --game <string>	path to the game (required)
-i, --iterations <number>	number of iterations to simulate (default: 1000000)
-r, --rngAlgorithm <string>	rng algorithm (isaac, mersenne-twister) (default: "isaac")
-c, --cores <number>	number of cores (default: "8")
-a, --startingAction <string>	action (default: "main")
-ia, --initialAction <string>	initial action
-b, --bet <number>	initial bet (default: 100)
-s, --strategy <string>	simulation strategy (for action and sideBets)
-v, --variant <string>	config variant
-h, --help	display help for command

Bet Limits

A set parameters specific for the player betting configuration. It allows for more sophisticated validation of player actions. E.g. it can be used to validate what player actions are allowed in order not to breach maxExposure specified by the operator.

`IBetLimits` object includes:

- `minBet` : min bet in player's currency
- `maxBet` : max bet in player's currency
- `maxBonusBet` : max bonus bet (for non- `main` bets) in player's currency
- `maxExposure` : max exposure in base in player's currency
- `currencyRate` : currency rate to base currency (typically `eur`)
- `exchangeRate` : exchange rate (from Currency feeds) to base currency (typically `eur`)
- `currencyDecimals` : amount of decimal places for the currency
- `currencyUnit` : minimal monetary value of a currency (e.g. 0.01 for "cent")